

第 6 章：DQN (Deep Q-Network, 深度 Q 网络) 与经典改进

6.1 本章符号说明

符号/缩写	English	中文	含义
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
$Q(s,a)$	Action-value Function	动作价值函数	状态 s 下动作 a 的长期价值； Q 表示 action quality。
$V(s)$	State-value Function	状态价值函数	Dueling DQN 中状态 s 本身的长期价值。
$A(s,a)$	Advantage Function	优势函数	动作 a 相对状态平均水平的额外价值；这里的斜体 A 不是动作集合。
$\mathcal{A} \mid \mathcal{A} $	Action Set / Number of Actions	动作集合 / 动作数量	$\mathcal{A}$ 是离散动作集合， $ \mathcal{A} $ 是其中动作的个数。
(s,a,r,s',d)	Transition Tuple	转移样本五元组	当前状态、动作、即时奖励、下一状态和终止指示量；代码常把 d 写成 <code>done</code> 。
$d \mid \text{done}$	Terminal Indicator	终止指示量	$d=1$ 表示真正终止，target 不再 bootstrap（不再接入下一状态的估计价值）； $d=0$ 表示可以继续估计未来价值。旧代码常把 d 命名为 <code>done</code> 。
γ	Discount Factor	折扣因子	控制下一状态价值在 TD target 中的权重。
α	Learning Rate / Step Size	学习率 / 步长	Q-learning 更新中吸收 TD error 的比例。
y	TD Target / Bellman Target	TD 目标 / Bellman 目标	用于监督当前 Q 网络的目标值。
$L(\theta)$	Loss Function	损失函数	网络参数 θ 的训练损失。
θ	Online Network Parameters	在线网络参数	当前正在训练的 Q 网络参数。
$\bar{\theta}$	Target Network Parameters	目标网络参数	滞后更新的 target network 参数。
p_i	Priority	优先级	Prioritized Replay 中第 i 条样本的采样优先级。
$P(i)$	Sampling Probability	采样概率	replay buffer 中样本 i 被采样的概率。
δ_i	TD Error of Sample i	第 i 条样本的 TD 误差	target y_i 与当前预测 $Q(s_i,a_i;\theta)$ 的差。
N	Buffer Size / Sample Count	buffer 大小 / 样本数	importance sampling weight 中的数据规模。
w_i	Importance Sampling Weight	重要性采样权重	修正非均匀采样 bias 的权重。

符号/缩写	English	中文	含义
α_p / β_{IS}	Priority Exponent / IS Correction Exponent	优先级指数 / 重要性采样修正指数	Prioritized Replay 中控制采样偏斜和 bias 修正的参数; α_p 与学习率 α 不是同一个量。
ϵ_p	Priority Floor	优先级下限常数	防止 TD error 为零的样本获得零采样概率的小正数; 与 epsilon-greedy 的探索率无关。
TD	Temporal Difference	时序差分	DQN 使用 TD/Bellman target 训练 Q 网络。
RL	Reinforcement Learning	强化学习	DQN 是 deep RL 的经典 value-based 算法。
SGD	Stochastic Gradient Descent	随机梯度下降	用 mini-batch 优化神经网络的训练方法。
IS	Importance Sampling	重要性采样	Prioritized Replay 中用于修正非均匀采样 bias。
DDPG / TD3 / SAC / PPO	Deep Deterministic Policy Gradient / Twin Delayed DDPG / Soft Actor-Critic / Proximal Policy Optimization	深度确定性策略梯度 / 双延迟 DDPG / 软 Actor-Critic / 近端策略优化	连续控制或策略优化章节会出现的算法。

6.2 本章目标

本章学习 value-based deep RL 的代表算法 DQN。你需要掌握：

- DQN 的基本结构。
- experience replay。
- target network。
- Double DQN。
- Dueling DQN。
- Prioritized Experience Replay。
- DQN 的适用场景和局限。

6.3 本章主线

本章把第 3 章的 Q-learning 和第 5 章的神经网络函数逼近接起来。直接用网络拟合 Q-learning target 会因为样本相关、target 移动、过估计等问题不稳定。DQN 的 replay buffer、target network、Double DQN、Dueling DQN 和 Prioritized Replay 都是在修正这些具体不稳定来源。

6.4 本章新增概念说明

DQN 章节名词很多，先按“基础结构 -> 稳定技巧 -> 改进算法”分清楚：

名词	中文	先记住什么
Q-table	Q 表	表格 Q-learning 中为每个 (s,a) 存一个值。
Q network	Q 网络	DQN 用神经网络输出 $Q(s,a)$ ，替代 Q-table。

名词	中文	先记住什么
online network	在线网络	当前正在训练、负责预测 Q 值的网络。
target network	目标网络	滞后于 online network，用来计算较稳定的 TD target。
replay buffer / experience replay	回放池 / 经验回放	存历史 transition，随机采样 mini-batch 训练。
TD target / Bellman target	TD 目标 / Bellman 目标	当前 Q 网络要拟合的监督信号，例如 $r + \gamma \max_{a'} Q(s', a')$ 。
bootstrap / bootstrapping	自举	target 中接入模型自己对下一状态的当前估计，例如 $r + \gamma V(s')$ ；好处是无需等完整 episode，风险是估计误差会传递。
Huber loss	Huber 损失	对异常 TD error 比平方损失更稳的回归损失。
overestimation bias	过估计偏差	对带噪声的多个 Q 估计取最大值时，更容易选中被正噪声高估的动作。
sample efficiency	样本效率	每条环境交互数据能被算法利用到什么程度；replay 复用旧样本可以提高样本效率。
Gymnasium / API	Python RL 环境库 / 应用程序编程接口	Gymnasium 是常用的 Python 强化学习环境库；API 是 Application Programming Interface（应用程序编程接口），规定代码如何调用环境以及环境返回哪些字段。
terminated / truncated	任务终止 / 外部截断	terminated 表示到达任务定义的终止状态；truncated 表示因时间上限等任务外条件停止。二者都会结束当前 episode，但不一定都应清零 bootstrap value。
stop gradient	停止梯度	把 TD target 当作固定监督标签，反向传播时不让梯度穿过 target 分支。
CleanRL	CleanRL 开源实现	一组强调单文件、便于阅读的强化学习算法实现；本章练习只把它作为代码阅读材料。
Double DQN	双网络 DQN 改进	online network 选动作，target network 估值，降低过估计。
Dueling DQN	Dueling 架构	把 $Q(s, a)$ 拆成状态价值 $V(s)$ 和动作优势 $A(s, a)$ 。
Prioritized Replay	优先经验回放	更常采样 TD error 大的样本。
importance sampling weight	重要性采样权重	修正 Prioritized Replay 非均匀采样带来的偏差。

6.5 DQN 基础

6.5.1 DQN 解决什么问题

Q-learning 的表格更新是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

当状态空间巨大时，无法维护 Q-table。DQN 用神经网络近似 Q 函数：

$$Q(s, a; \theta)$$

如果动作空间是离散的，网络通常输入 s ，输出每个 action 的 Q 值：

$$\text{network}(s) \rightarrow [Q(s, a_1), Q(s, a_2), \dots, Q(s, a_n)]$$

6.5.2 DQN Loss

DQN 的 transition 记为 (s, a, r, s', d) ，其中 $d \in \{0, 1\}$ 。统一写成带终止掩码的 target：

$$y = r + \gamma(1-d) \max_{a'} Q(s', a'; \theta)$$

如果 s' 是终止状态， $d=1$ ，上式自动退化为：

$$y = r$$

损失函数：

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2]$$

其中：

- θ 是 online network 参数。
- θ 是 target network 参数。

实际实现中常用 Huber loss：

$$L = \text{Huber}(y - Q(s, a; \theta))$$

Huber loss 对异常 TD error 更稳。

6.5.3 从 Q-learning 到 DQN Loss 的推理过程

Tabular Q-learning 的更新是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha[y - Q(s, a)]$$

其中：

$$y = r + \gamma(1-d) \max_{a'} Q(s', a')$$

如果用神经网络 $Q(s, a; \theta)$ 代替表格，就不能直接“修改表格里的一個格子”。我们能做的是让网络当前输出靠近 target y 。于是 RL 更新变成一个监督学习式回归问题：

```
input:  (s, a)
label:  y = r + gamma (1 - done) max Q_target(s', a')
predict: Q(s, a; theta)
loss:   (y - Q(s, a; theta))^2
```

这一步很关键：DQN 不是突然换了目标，它仍然在逼近 Bellman optimality backup，只是把 tabular update 改成了 neural network regression。

为什么 target 用 θ^- 而不是当前 θ ?

- 如果 label 和 prediction 都由同一个快速变化的网络产生，训练目标会一直漂。
- target network 让 y 在一段时间内相对稳定。
- online network 只负责追这个相对固定的 Bellman target。

面试表达：

DQN 的 loss 可以看成把 Q-learning 的 TD target 当成监督信号。Replay buffer 解决样本相关性，target network 解决 bootstrap target 随参数同步漂移的问题。

6.5.4 DQN 训练流程

```
initialize online network Q(theta)
initialize target network Q(theta-) = Q(theta)
initialize replay buffer

for each step:
    choose action using epsilon-greedy
    execute action, observe r, s', terminated, truncated
    d = 1 if terminated else 0
    store (s, a, r, s', d) in replay buffer

    sample minibatch from replay buffer
    y = r + gamma (1 - d) max_a' Q(s', a'; theta-)
    minimize [y - Q(s, a; theta)]^2

    periodically update target network
```

6.6 DQN 稳定组件

6.6.1 Experience Replay

Replay buffer 存储 transition:

```
(s, a, r, s', d)
```

字段逐一解释：

字段	含义
s	执行动作之前的当前状态。
a	agent 在 s 中实际执行的动作。
r	执行动作后环境返回的即时奖励。
s'	环境转移到的下一状态。
d / done	训练公式使用的终止掩码；真正终止时为 1，否则为 0。done 是许多旧实现使用的字段名。

Gymnasium 到底是什么？

Gymnasium 是一个常用的 Python 强化学习环境库。它提供统一的环境调用方式，让预置的教学环境和自定义环境都可以通过类似代码交互。API 是 **Application Programming Interface** (应用程序编程接口)；这里具体指 `env.step(action)` 的返回约定：

```
(
    next_state,
    reward,
    terminated,
    truncated,
    info,
) = env.step(action)
```

返回字段	含义与训练影响
<code>next_state</code>	执行动作后的下一状态，对应公式中的 s' 。
<code>reward</code>	本步即时奖励，对应公式中的 r 。
<code>terminated</code>	到达任务本身定义的终止状态，例如杆已经倒下；通常令训练掩码 $d=1$ ，不再接入下一状态价值。
<code>truncated</code>	因时间上限等外部条件停止，而非任务自然终止；当前 episode 要结束，但若最终观测仍是有效状态，通常令 $d=0$ 并继续 bootstrap。
<code>info</code>	环境附带的调试或统计信息，通常不直接进入 DQN target。

因此，“是否需要重置环境”和“是否清零未来价值”是两个问题：

```
episode_finished = terminated or truncated # 是否 reset 环境
d = float(terminated) # 是否清零 bootstrap value
```

为避免把二者混淆，实际 replay buffer 最稳妥的做法是分别保存 `terminated` 和 `truncated`，构造 target 时再按任务语义生成 d 。本课程公式继续使用简洁的五元组 (s, a, r, s', d) 。

作用：

- 打破连续样本之间的相关性。
- 复用历史样本，提高 sample efficiency。
- 平滑训练数据分布。

面试表达：

DQN 中连续交互得到的样本高度相关，不适合直接做 SGD。Replay buffer 通过随机采样历史 transition，降低样本相关性，同时让每条经验被多次利用。

6.6.2 Target Network

如果 target 使用 online network：

$$y = r + \gamma(1-d) \max_{a'} Q(s', a'; \theta)$$

那么 θ 每次更新都会改变 target，训练会追逐一个移动目标。

DQN 使用 target network：

$$y = r + \gamma(1-d) \max_a Q(s', a'; \theta)$$

θ 每隔固定步数从 θ 同步一次, 或使用软更新。

面试表达:

Target network 让 bootstrap target 在一段时间内保持稳定, 降低训练目标和预测网络同步变化带来的震荡。

6.7 DQN 经典改进

6.7.1 Double DQN

DQN 中的 target:

$$\max_a Q(s', a'; \theta)$$

同一个 max 操作既选择动作又评估动作, 容易产生 overestimation bias。

Double DQN 将动作选择和动作评估拆开:

$$\begin{aligned} a^* &= \arg \max_a Q(s', a'; \theta) \\ y &= r + \gamma(1-d) Q(s', a^*; \theta) \end{aligned}$$

其中:

- online network 负责选择动作。
- target network 负责评估该动作。

这样可以缓解 Q 值过估计。

6.7.2 Double DQN 为什么能缓解过估计

DQN 的问题来自 max operator。假设每个动作的 Q 估计都有噪声:

$$\text{estimated } Q = \text{true } Q + \text{noise}$$

当我们取 $\max_a Q(s', a)$ 时, 更容易选中“被正噪声高估”的动作。即使每个动作估计平均无偏, 最大值也会偏高。

普通 DQN 同一个 target network 同时做两件事:

1. 选择哪个动作最大。
2. 评估这个动作的价值。

Double DQN 把这两步拆开:

$$a^* = \arg \max_a Q(s', a'; \theta)$$

$$y = r + \gamma(1-d) Q(s', a^*; \theta)$$

这样选择动作时的噪声和评估动作时的噪声不完全相同, 正噪声被重复放大的程度下降, 因此 overestimation bias 更小。

面试表达:

Double DQN 的核心是 decouple action selection and action evaluation。DQN 的 max operator 容易选择被噪声高估的动作，Double DQN 用 online network 选动作，用 target network 估值，从而降低 overestimation bias。

6.7.3 Dueling DQN

Dueling DQN 将 Q 函数分解为状态价值和动作优势：

$$Q(s, a) = V(s) + A(s, a)$$

这里的斜体 $A(s, a)$ 是 **Advantage Function (优势函数)**：它回答“在状态 s 下，动作 a 比该状态的平均动作好多少”。花体 $\mathcal{A}$ 才是 **Action Set (动作集合)**， $|\mathcal{A}|$ 是离散动作数量。

直接写 $Q=V+A$ 不可辨识：任取常数 c ，把 $V(s)$ 改成 $V(s)+c$ ，同时把所有 $A(s, a)$ 改成 $A(s, a)-c$ ，得到的 $Q(s, a)$ 完全不变。网络因此无法唯一确定哪部分该归给 V 、哪部分该归给 A 。

实际常把同一状态下所有动作的 advantage 减去均值：

$$Q(s, a) = V(s) + A(s, a) - 1 / |\mathcal{A}| \sum_{a'} A(s, a')$$

减去均值后，聚合层使用的是“相对优势”，从而消除任意整体平移。这里 a' 只是遍历动作集合 $\mathcal{A}$ 的求和下标。

数值例子：某状态的 $V(s)=5$ ，两个动作的原始 advantage 分别是 $[2, 0]$ ，均值为 1。聚合后的 Q 值是：

$$\begin{aligned} Q(s, \text{left}) &= 5 + 2 - 1 = 6 \\ Q(s, \text{right}) &= 5 + 0 - 1 = 4 \end{aligned}$$

所以 $V(s)=5$ 表示状态整体水平，动作的中心化 advantage $[1, -1]$ 决定两个动作相对这个水平的上下浮动。

直觉：

有些状态下，知道状态本身好不好比区分每个动作更重要。Dueling 结构让网络分别学习：

- $V(s)$ ：这个状态整体价值如何。
- $A(s, a)$ ：某个动作相对其他动作好多少。

适合动作之间差异不总是明显的场景。

6.7.4 Prioritized Experience Replay

普通 replay buffer 均匀采样 transition。

Prioritized Replay 更倾向采样 TD error 大的样本。

优先级：

$$\begin{aligned} \delta_i &= y_i - Q(s_i, a_i; \theta) \\ p_i &= |\delta_i| + \epsilon_p \end{aligned}$$

采样概率：

$$P(i) = p_i^{\alpha_p} / \sum_k p_k^{\alpha_p}$$

为了修正非均匀采样带来的 bias，使用 importance sampling weight：

$$w_i = (1 / (N P(i)))^{\beta_{IS}}$$

其中 δ_i 是第 i 条样本的 TD error, $\varepsilon_p > 0$ 防止零优先级, α_p 控制采样偏向程度, β_{IS} 控制 importance sampling 修正强度; 求和下标 k 遍历 buffer 中的候选样本。实现中常再用 batch 内最大权重归一化 w_i , 避免梯度尺度突然变大。

直觉:

TD error 大的样本说明当前模型学得不好, 更值得训练。

6.7.5 Rainbow DQN

Rainbow DQN 组合了多个 DQN 改进:

- Double DQN: 拆开动作选择和动作估值, 缓解过估计。
- Dueling Network: 把 Q 分成状态价值和动作优势两条分支。
- Prioritized Replay: 更常抽取 TD error 大的 transition。
- Multi-step Learning (多步学习): 用连续多步真实 reward 加末端 bootstrap value 构造 target, 让奖励更快向前传播。
- Distributional RL (分布强化学习): 不只预测 return 的期望, 而是预测 return 的概率分布。
- NoisyNet (噪声网络): 给网络参数加入可学习噪声, 让策略通过参数扰动进行探索。

面试中通常了解组件即可, 不一定要完整推导。

6.8 DQN 适用场景和例子

6.8.1 DQN 的适用场景

适合:

- 离散动作空间。
- 可以通过 Q 值枚举动作。
- 游戏、简单控制、离散决策。

不适合:

- 高维连续动作空间。
- 需要直接输出连续控制量的任务。
- 极端稀疏奖励且探索困难的任务。

连续控制通常用 DDPG、TD3、SAC、PPO 等算法。

6.8.2 本章例子: 从 CartPole 到 DQN

6.8.2.1 例子 1: CartPole

CartPole (小车倒立摆) 是一个经典控制基准: agent 左右推动小车, 目标是让上方的杆尽量保持直立。它的状态是:

状态维度	含义
cart position	小车位置
cart velocity	小车速度
pole angle	杆角度
pole angular velocity	杆角速度

动作只有两个:

向左推 / 向右推

这类任务可以用 DQN 学：

输入状态 s
 输出 $Q(s, \text{left}), Q(s, \text{right})$
 选择 Q 最大的动作

6.8.2.2 例子 2：为什么需要 replay buffer

如果 agent 连续玩游戏，连续样本高度相关：

第 t 帧：球在左边
 第 $t+1$ 帧：球稍微往右
 第 $t+2$ 帧：球继续往右

直接按时间顺序训练，网络容易被最近局部轨迹带偏。Replay buffer 随机采样历史 transition，让 mini-batch 更接近独立同分布。

6.8.2.3 例子 3：Double DQN 过估计小例子

假设真实下一状态两个动作价值都接近 10，但网络估计有噪声：

$Q(s', a1) = 10.1$
 $Q(s', a2) = 9.8$

普通 DQN 用同一个网络选动作和估值，会倾向选被噪声高估的动作。Double DQN 把“选动作”和“估值”拆开，可以降低这种系统性过估计。

面试表达：

DQN 的关键不是“神经网络版 Q-learning”这么简单，而是 replay buffer 和 target network 解决了深度网络训练时的相关样本和移动目标问题。

6.9 本章面试高频问题

6.9.1 DQN 相比 Q-learning 改了什么？

DQN 用神经网络近似 Q 函数，使 Q-learning 可以处理高维状态输入，例如图像。为了稳定训练，引入了 experience replay 和 target network。

6.9.2 DQN 为什么需要 replay buffer？

为了打破连续样本相关性，提高样本利用率，并使 mini-batch 训练更接近监督学习中的随机采样。

6.9.3 DQN 为什么需要 target network？

为了稳定 bootstrap target。如果 target 和 online prediction 使用同一网络，模型每次更新都会改变训练目标，容易震荡或发散。

6.9.4 Double DQN 解决什么问题？

解决 DQN 中 max operator 带来的 overestimation bias。它用 online network 选择动作，用 target network 评估动作，从而降低过估计。

6.9.5 Dueling DQN 的核心思想是什么？

将 Q 分解为状态价值 $V(s)$ 和动作优势 $A(s,a)$ ，让网络更好地学习状态本身价值和动作相对价值。

6.10 本章练习

1. 手写 DQN target 和 loss。
2. 解释 Double DQN 的 target 和 DQN target 的区别。
3. 画出 Dueling DQN 的网络结构。
4. 阅读一个 CleanRL (便于阅读的单文件 RL 开源实现集合) DQN 实现，标出 replay buffer、target network、epsilon schedule 的位置。

▼ 参考答案 (点击展开)

1. **DQN target 和 loss**: 统一 target 是 $y=r+\gamma(1-d)\max_a Q(s',a';\theta)$; $d=1$ 时自动变成 $y=r$ 。常用损失为 $L(\theta)=\mathbb{E}[(y-Q(s,a;\theta))^2]$ 或 Huber loss; 计算 y 时需要 stop gradient (停止梯度), 即把 target 当作固定标签。
2. **Double DQN 的区别**: 普通 DQN 用 target network 同时选动作和估值: $y=r+\gamma(1-d)\max_a Q(s',a';\theta)$ 。Double DQN 用 online network 选 $a=\arg \max_a Q(s',a';\theta)$, 再用 target network 估值: $y=r+\gamma(1-d)Q(s',a';\theta)$, 从而解耦 selection 和 evaluation。
3. **Dueling DQN 结构**:

```
state -> shared encoder -> state-value head V(s) -----+
                                     -> advantage head A(s, a) -----+--> aggregate -> Q(s, a)

Q(s, a) = V(s) + A(s, a) - mean_a' A(s, a')
```

减去平均 advantage 用于消除 V 与 A 分解的不唯一性。

4. **CleanRL 阅读检查点**: replay buffer 通常在初始化区创建, 在环境交互后调用 `add`, 训练时调用 `sample`; target network 在 online network 之后创建, 按固定频率复制参数; epsilon schedule (探索率随训练进度变化的规则) 在选动作前根据 `global_step` (累计环境交互步数) 计算, 随机数小于 epsilon 时采样随机动作, 否则用 Q 网络取 `argmax`。阅读时还应检查 terminal mask (终止掩码 d)、learning-starts (开始梯度更新前至少积累多少 transition) 和 target-update frequency (目标网络同步间隔)。